

PyTorch CHEAT SHEET

1 Load data 2 Define model 3 Train model 4 Evaluate model

General

PyTorch is a open source machine learning framework. It uses **torch.Tensor** – multi-dimensional matrices – to process. A core feature of neural networks in PyTorch is the autograd package, which provides automatic derivative calculations for all operations on tensors.

import torch	Root package	torch.randn(*size)	Create random tensor
import torch.nn as nn	Neural networks	torch.Tensor(L)	Create tensor from list
from torchvision import datasets, models, transforms	Popular image datasets, architectures & transforms	tnsr.view(a,b, ...)	Reshape tensor to size (a, b, ...)
import torch.nn.functional as F	Collection of layers, activations & more	requires_grad=True	tracks computation history for derivative calculations

Layers

 nn.Linear(m, n): Fully Connected layer (or dense layer) from m to n neurons	 nn.ConvXd(m, n, s): X-dimensional convolutional layer from m to n channels with kernel size s; $X \in \{1, 2, 3\}$
 nn.Flatten(): Flattens a contiguous range of dimensions into a tensor	 nn.MaxPoolXd(s): X-dimensional pooling layer with kernel size s; $X \in \{1, 2, 3\}$
 nn.Dropout(p=0.5): Randomly sets input elements to zero during training to prevent overfitting	 nn.BatchNormXd(n): Normalizes a X-dimensional input batch with n features; $X \in \{1, 2, 3\}$
 nn.Embedding(m, n): Lookup table to map dictionary of size m to embedding vector of size n	 nn.RNN/LSTM/GRU: Recurrent networks connect neurons of one layer with neurons of the same or a previous layer

*torch.nn offers a bunch of other building blocks.
A list of state-of-the-art architectures can be found at <https://paperswithcode.com/sota>.*

Load data

A dataset is represented by a class that inherits from **Dataset** (resembles a list of tuples of the form (features, label)).

DataLoader allows to load a dataset without caring about its structure.

Usually the dataset is split into training (e.g. 80%) and test data (e.g. 20%).

```
1 from torch.utils.data
2 import Dataset, TensorDataset,
3     DataLoader, random_split
4
5 train_data, test_data =
6     random_split(
7         TensorDataset(inps, tgts),
8         [train_size, test_size]
9     )
10
11 train_loader =
12     DataLoader(
13         dataset=train_data,
14         batch_size=16,
15         shuffle=True)
```

Activation functions

Common activation functions include **ReLU**, **Sigmoid** and **Tanh**, but there are other activation functions as well.

nn.ReLU() creates a nn.Module for example to be used in Sequential models. **F.relu()** ist just a call of the ReLU function e.g. to be used in the forward method.

 nn.ReLU() or F.relu() Output between 0 and ∞ , most frequently used activation function	 nn.Sigmoid() or F.sigmoid() Output between 0 and 1, often used for predicting probabilities
 nn.Tanh() or F.tanh() Output between -1 and 1, often used for classification with two classes	

Define model

There are several ways to define a neural network in PyTorch, e.g. with **nn.Sequential** (a), as a class (b) or using a combination of both.

```
model = nn.Sequential(
    nn.Conv2D(1, 1, 1)
    nn.ReLU()
    nn.MaxPool2D(1)
    nn.Flatten()
    nn.Linear(1, 1)
)
```

```
class Net(nn.Module):
    def __init__():
        super(Net, self).__init__()

        self.conv
            = nn.Conv2D(1, 1, 1)

        self.pool
            = nn.MaxPool2D(1)

        self.fc = nn.Linear(1, 1)

    def forward(self, x):
        x = self.pool(
            F.relu(self.conv(x))
        )

        x = x.view(-1, 1)

        x = self.fc(x)

        return x
model = Net()
```

Train model

LOSS FUNCTIONS

PyTorch already offers a bunch of different loss fuctions, e.g.:

nn.L1Loss	Mean absolute error
nn.MSELoss	Mean squared error (L2Loss)
nn.CrossEntropyLoss	Cross entropy, e.g. for single-label classification or unbalanced training set
nn.BCELoss	Binary cross entropy, e.g. for multi-label classification or autoencoders

OPTIMIZATION (torch.optim)

Optimization algorithms are used to update weights and dynamically adapt the learning rate with gradient descent, e.g.:

optim.SGD	Stochastic gradient descent
optim.Adam	Adaptive moment estimation
optim.Adagrad	Adaptive gradient
optim.RMSProp	Root mean square prop

```
1 correct = 0 # correctly classified
2 total = 0 # classified in total
3
4 model.eval()
5 with torch.no_grad():
6     for data in test_loader:
7         inputs, labels = data
8         outputs = model(inputs)
9         _, predicted = torch.max(outputs.data, 1)
10        total += labels.size(0) # batch size
11        correct += (predicted==labels)
12                               .sum().item()
13
14 print('Accuracy: %s' % (correct/total))
```

Save/Load model

model = torch.load('PATH') Load model
torch.save(model, 'PATH') Save model

It is common practice to save only the model parameters, not the whole model using model.state_dict()

```
1 torch.save(model.state_dict(), 'params.ckpt')
2 model.load_state_dict(
3     torch.load('params.ckpt'))
```

GPU Training

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

If a GPU with CUDA support is available, computations are sent to the GPU with ID 0 using **model.to(device)** or **inputs, labels = data[0].to(device), data[1].to(device)**.

```
1 import torch.optim as optim
2
3 # Define loss function
4 loss_fn = nn.CrossEntropyLoss()
5
6 # Choose optimization method
7 optimizer = optim.SGD(model.parameters(),
8                        lr=0.001, momentum=0.9)
9
10 # Loop over dataset multiple times (epochs)
11 for epoch in range(2):
12     model.train() # activate training mode
13     for i, data in enumerate(train_loader, 0):
14         # data is a batch of [inputs, labels]
15         inputs, labels = data
16
17         # zero gradients
18         optimizer.zero_grad()
19
20         # calculate outputs
21         outputs = model(inputs)
22         # calculate loss & backpropagate error
23         loss = loss_fn(outputs, labels)
24         loss.backward()
25         # update weights & learning rate
26         optimizer.step()
```

Evaluate model

The evaluation examines whether the model provides satisfactory results on previously withheld data. Depending on the objective, different metrics are used, such as accuracy, precision, recall, F1, or BLEU.

model.eval() Activates evaluation mode, some layers behave differently
torch.no_grad() Prevents tracking history, reduces memory usage, speeds up calculations